
Practical Laboratory

Lab Scenario

You will continue working with the **members** table created in previous labs, then create additional tables (**suppliers, products, and orders**) to practice joins and database maintenance operations.

Part A: Modifying the Table

1. Add a new column called `email` (`TEXT`).
 2. Add a `CHECK` constraint ensuring `age` is at least 0 if provided.
 3. Rename the `address` column to `city`.
 4. Drop the `email` column again.
-

Part B: Sorting, Aliases, and DISTINCT

5. List all members ordered by age from oldest to youngest.
 6. List all members ordered by city, then by `std_name`.
 7. List all members with `NULL` ages appearing last.
 8. Display `std_name` using the alias "**member name**".
 9. Concatenate `std_name` and `city` into a single readable column.
 10. Display the distinct list of cities.
 11. Use a table alias to display `std_id` and `std_name` for members with `std_id > 5`.
-

Part C: Aggregate Functions and Pagination

12. Count how many members live in each city.
13. Find the average age per city.
14. Display only cities with more than one member using `HAVING`.
15. Find the minimum and maximum salary among all members.
16. Display the first 5 members ordered by `std_id`.
17. Display the next 5 members using `OFFSET`.
18. Calculate the `OFFSET` required to display page 4 with a page size of 5.

Part D: SQL Essentials

19. Write a `SELECT` statement without a semicolon and observe what `psql` does.
20. Write the same column alias twice: once using `AS` and once without `AS`.
21. Concatenate two text values using `||` and verify that using `+` produces an error.

Part E: Creating New Tables

22. Create a `suppliers` table with:

- `supplier_id SERIAL PRIMARY KEY`
- `name TEXT`

23. Insert three suppliers without specifying the IDs and verify that the IDs increment automatically.

24. Create the `products` and `orders` tables.

25. Insert:

- At least **4 products**.
- At least **3 orders**.
- Leave at least **one product** with no orders.

Part F: Joins

26. Write an `INNER JOIN` to list each order with its product name.

27. Write a `LEFT JOIN` to list every product, including products that have never been ordered.

28. Write a `FULL JOIN` and describe in a comment what additional rows it may return compared to a `LEFT JOIN`.

29. Write a `CROSS JOIN` between `products` and `orders` and verify that the number of rows equals:

```
rows(products) × rows(orders)
```

Part G: DELETE, TRUNCATE, and DROP

30. Delete a single row from `orders` using `WHERE` and verify the `DELETE 1` message.

31. Truncate the `orders` table and verify that it is empty but still exists.
 32. Drop the `orders` table and confirm that it no longer appears in `\dt`.
-

Lab Safety Exercise

Before running an `UPDATE`, `DELETE`, `TRUNCATE`, or `DROP` command on a shared table:

- Confirm the affected rows with an appropriate `SELECT` statement.
- Always verify the command with your instructor or team before executing it.

Expected answer: `UPDATE` or `DELETE` without a `WHERE` clause affects every row in the table. `TRUNCATE` and `DROP` remove data without allowing filtering, so these operations can result in accidental data loss if performed without verification.

Practical Challenge

Write a query that lists every product together with:

- The number of times it has been ordered.
- The average order quantity (or another appropriate aggregate if available).

Requirements:

- Use `LEFT JOIN`.
- Group the results by product.
- Include products that have never been ordered.
- Display only products ordered **two or more times**.
- Sort the results by the number of orders in descending order.